

01 Introduction to Functions

In programming, a **function** is a reusable block of code designed to perform a specific task. Instead of writing the same instructions multiple times, you group them into a function and execute them whenever needed.

Why Functions Are Useful:

- **Code Reusability:** Write once, use multiple times without repeating code.
- **Clarity & Structure:** Breaks large programs into neat, manageable chunks.
- **Easier Maintenance:** Fixing a bug in a function updates it everywhere across your program.

What is a Built-in Function?

Python comes with a wide variety of standard functions that are permanently available out-of-the-box. You do not need to install extra libraries or write custom code before using them.

Built-in vs. Programmer-Defined Functions

Built-in Functions: Pre-packaged with Python (e.g., `print()`, `len()`, `input()`).

User-Defined Functions: Created by programmers using the `def` keyword to perform custom tasks.

Calling a Function:

Executing a function is known as *calling* or *invoking* it. You call a function by writing its name followed by parentheses `()`. Any value passed inside the parentheses is called an **argument**.

```
print("Hello Python")
```

DECONSTRUCTION

- **Function Name:** `print`
- **Argument Provided:** `"Hello Python"` (a string literal)

02 The print() Function

Purpose: Outputs text, numbers, variables, or expressions directly to the screen console.

Syntax: `print(value1, value2, ..., sep=' ', end='\n')`

```
# 1. Basic output print("Hello") # 2. Printing multiple values (separated by default space) print("Python", "Programming") # 3. Custom separator using 'sep' parameter print("A", "B", "C", sep="-") # 4. Custom line ending using 'end' parameter print("Hello", end=" ") print("World")
```

EXPECTED OUTPUT

```
Hello
Python Programming
A-B-C
Hello World
```

03 The input() Function

Purpose: Pauses program execution and reads a line of text entered by the user from the keyboard.

CRITICAL RULE: The `input()` function always returns data as a string (`str`), even if the user enters digits!

04 Type Conversion – int()

Purpose: Converts a compatible value (such as a string of digits or a float) into a whole integer number.

Since `input()` yields a string, you must wrap it inside `int()` to perform numerical calculations.

```
age = int(input("Enter your age: ")) next_year = age + 1 print("Next year you will be:", next_year)
```

CONSOLE INTERACTION

```
Enter your age: 16
Next year you will be: 17
```

05 Type Conversion – float()

Purpose: Converts integers or valid numeric strings into floating-point (decimal) numbers.

```
price = float(input("Enter price: ")) tax = price * 0.05 print("Tax amount:", tax)
```

CONSOLE INTERACTION

```
Enter price: 99.50
Tax amount: 4.975
```

06 Type Conversion – str()

Purpose: Converts any data type (integers, floats, lists) into its string text representation.

This is extremely useful when concatenating strings with numeric variables using the `+` operator.

```
age = 17 # Combining string and integer using str() message = "I am " + str(age) + " years old." print(message)
```

EXPECTED OUTPUT

```
I am 17 years old.
```

07 The len() Function

Purpose: Returns the total number of characters in a string or items in a collection.

```
text = "Python" print(len(text)) phrase = "Hello World" print(len(phrase)) # Blank spaces count as characters!
```

EXPECTED OUTPUT

```
6
11
```

08 The type() Function

Purpose: Inspects and displays the exact data type classification of any variable or value.

```
print(type(10)) print(type(3.14)) print(type("Python"))
```

10 The round() Function

Purpose: Rounds a floating-point number to a specified number of decimal digits.

If the second argument is omitted, it rounds the number to the nearest integer.

```
print(round(3.14159)) # Rounds to nearest integer print(round(3.14159, 2)) # Rounds to 2 decimal places
```

EXPECTED OUTPUT

```
3
3.14
```

11 , 12 , 13 , 14 Sequence Functions: max(), min(), sum(), sorted()

These built-in functions allow us to quickly analyze and manipulate ordered groups of values (such as lists of numbers):

```
numbers = [10, 25, 7, 40] print(max(numbers)) # Returns largest element print(min(numbers)) # Returns smallest element print(sum(numbers)) # Adds all numbers together print(sorted(numbers)) # Returns a fresh list in ascending order
```

EXPECTED OUTPUT

```
40
7
82
[7, 10, 25, 40]
```

15 Built-in Functions Quick Reference

Function	Purpose	Example
<code>print()</code>	Displays output to console	<code>print("Hi")</code>
<code>input()</code>	Takes user string input	<code>input("Name: ")</code>
<code>int()</code>	Converts to integer	<code>int("25")</code>
<code>float()</code>	Converts to decimal float	<code>float("3.14")</code>
<code>str()</code>	Converts value to string	<code>str(100)</code>
<code>len()</code>	Finds length / item count	<code>len("Code")</code>
<code>type()</code>	Returns data type	<code>type(5.5)</code>
<code>abs()</code>	Calculates absolute value	<code>abs(-8)</code>
<code>round()</code>	Rounds decimal value	<code>round(4.6)</code>
<code>max()</code>	Finds largest item	<code>max([3, 9, 2])</code>
<code>min()</code>	Finds smallest item	<code>min([3, 9, 2])</code>
<code>sum()</code>	Adds numeric items	<code>sum([1, 2, 3])</code>
<code>sorted()</code>	Returns sorted collection	<code>sorted([3, 1, 2])</code>

16 Combined Practical Example

Let's combine input, type casting, string length, and data typing in one cohesive script:

17 Core Syntax Rules to Remember

- **Case Sensitivity:** Python is strict! `print()` works, but `Print()` causes a `NameError`.
- **Parentheses Required:** Always use parentheses `()` when calling a function.
- **Arguments Inside:** Values passed to a function must go inside the parentheses.
- **Input Defaults to String:** `input()` always produces a string. Convert numeric inputs explicitly.
- **Always Available:** Built-in functions require no `import` statements.

18 Common Beginner Mistakes & Corrections

✗ Mistake 1: Unconverted Input

```
# Fails during math! a = input
("Num: ") print(a + 5)
```

✓ Correction

```
a = int(input("Num: ")) print(a
+ 5)
```

✗ Mistake 2: Missing Parentheses

```
# Missing parentheses print "He
llo"
```

✓ Correction

```
print("Hello")
```

✗ Mistake 3: `len()` on Integer

```
# Int has no length! num = 250
print(len(num))
```

✓ Correction

```
num_str = "250" print(len(num_s
tr))
```

✗ Mistake 4: Capitalization Error

```
# Capitalized name x = Len("Cod
e")
```

✓ Correction

```
x = len("Code")
```

19 Practice Exercises

- Q1. Write a program to prompt users for their name and print a friendly welcome message.
- Q2. Prompt for two whole numbers, convert them using `int()`, and print their total sum.
- Q3. Accept a city name from the user and display the total number of letters in it.
- Q4. Given `scores = [88, 92, 79, 95]`, write statements to display the highest score and lowest score.
- Q5. Prompt for a decimal price (e.g., `19.876`) and display it rounded to 2 decimal places.

20 Mini Challenge — Student Information Program